An asynchronous spiking neural network which can learn temporal sequences

Joy Bose, S.B Furber, M. Cumpstey

School of Computer Science, University of Manchester, M13 9PL, UK
Email: {bosej@cs.manchester.ac.uk, sfurber@manchester.ac.uk, cumpstem@cs.manchester.ac.uk }

***Abstract*—We describe the design of an asynchronous spiking neural network that can learn and predict temporal sequences online. We concentrate on issues regarding the asynchronous functioning of the model such as timing relations between different autonomous components of the system.**

I. PROBLEM SPECIFICATION

Our aim is to implement a memory in spiking neurons that can learn any given number of sequences online (a sequence machine) in a single pass or single presentation of the sequence, and predict any learnt sequence correctly. A sequence is a series of symbols in temporal order, such as 'abc'.

The high-level description of the system (the functionality which is to be implemented in spiking neurons) is as follows: it takes as input a series of symbols constituting an input sequence, and for each input symbol it outputs a symbol which is the prediction for what the next symbol should be. If the input symbol is not part of a sequence previously learnt by the machine, the prediction will be incorrect but the machine will learn to predict correctly the next time the same sequence is presented.

Clearly, the prediction of the next symbol depends on the history of the sequence as well as the input symbol presented. In a system with infinite memory, the machine would be able to look as far back in the history as needed to produce an unambiguous prediction. However, we are using a finite neural memory which learns (writes to the memory) to associate the context or history of the sequence with the input, and so some noise is expected. The context or history itself is represented as a finite state machine, in which the new history is a function of the old history and the present input.

For example, if the sequence learnt is 'abcbd', the grammar learnt by the system can be represented as follows:

(Starting symbol) S → a

a → b	c → b	b → d
ab → c	bc → b	cb → d
abc → b	bcb → d	
abcb → d		

In this high-level description of the system, all the steps are assumed to take place in a perfectly synchronised way. However, in this paper we are interested in implementing this functionality using spiking neurons, which are essentially asynchronous, to get some insights on engineering and modelling issues in similar systems, as well as throw some light on the dynamics of interactions between biological neurons.

II. SPIKING NEURONS

A spiking neuron is a simplified model of a biological neuron that fires spikes or electrical impulses if an internal quantity of the neuron known as the activation exceeds a threshold. The activation is increased every time a spike fires at an input of the neuron, thus a neuron can be thought to accumulate input spikes. All spikes are of the same shape and information conveyed is only in the time of their firing. We assume that each spiking neuron fires only in response to its input spikes, i.e. there are no global control variables in the system that apply to all neurons. The neurons form layers, each layer performing a specific function and being connected to other layers, the spikes being transferred through the connection wires which may have different connection strengths, but we assume no wire delays.

III. SIMILARITY WITH BETWEEN A SPIKING NEURAL NETWORK AND ASYNC LOGIC CIRCUIT

In asynchronous logic design, communication takes place by transmission of electrical signals through wires, which can be considered similar to transmission of spikes in neural models. The electrical signals transmitted are in one of the two levels 0 and 1 (following binary logic), and the switching of levels could be considered as an event similar to firing a spike.

A standard asynchronous logic circuit has the handshake as its defining component. If we consider groups of spiking neurons interacting with each other, they show interactions similar to handshaking, in the sense that they can excite each other to generate corresponding bursts of spikes, which may be thought as the 'request' and 'acknowledge' signals. A latch, a standard component in asynchronous logic, can be implemented by a pair of spiking neurons which excite each other to fire a spike which keeps oscillating between them. The stored spike is released with the help of a third neuron that acts as a gate and resets the pair of neurons on receiving a 'request' control spike from another neuron. The released

spike can be considered as the 'acknowledge' signal in response to the 'request' signal.

IV. IMPLEMENTING THE SYSTEM USING NEURONS

In the high-level specification of the sequence machine, input symbols are associated with the context of the sequence and a prediction of the next symbol is generated. In the neural implementation, these symbols are encoded as bursts of spikes fired by layers of neurons. These spike bursts propagate like a wave through different layers of the system, each layer generating an output burst after receiving its input burst from the previous layer. The operation of the system is asynchronous, because there is no global mechanism such as a clock to synchronise the firing times of spikes and bursts of spikes across different layers.

In our system, we use a coding scheme known as rank ordered N-of-M code, in which we specify that N out of a total of M neurons in the layer can fire spikes in a burst, and the choice of the N firing neurons as well as the time order of their firing determines the code. The N-of-M code can be implemented by having a neuron that takes inputs from the M outputs of the layer and fires a resetting spike when N output spikes have fired in that layer that resets all the neurons. A neuron can be sensitised to a specific input firing order by multiplicatively decreasing the effect on activation increase for each successive input spike, and keeping the threshold of the neuron such that it fires when it has received the specific code. Finer details of the implementation of such a code using spiking neurons can be found elsewhere [2].

We use the wheel or spin model of the neuron in our implementation, in which the neuron can be visualised as a wheel spinning at a constant rate. The neuron has a quantity called activation or phase, which keeps on increasing at a constant speed, unless the neuron gets an input spike, which increases its activation (or phase) by an amount corresponding to the connection weight of the input neuron. The activation increases linearly till it reaches the threshold, which it will eventually, even if it gets no input spikes.

V. COMPONENTS OF THE SYSTEM

The system consists of the following neural layers as components: input, encoder, context, delay, address decoder, data store and output. Figure 1 shows the different component neural layers in the network and the connections between the components.

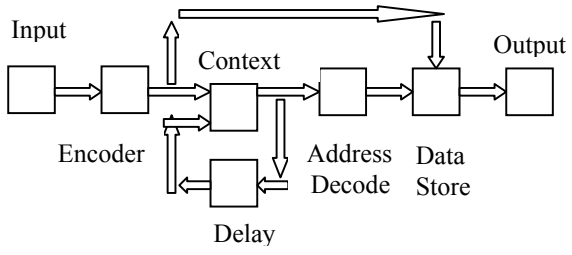

Fig. 1. Component neural layers of the sequence machine.

Most of these layers have fixed connection weights at their inputs and are essentially lookup tables, except for the data store (which is where the associations are written) and the context, which is like a finite state machine. We shall not mention here the detailed implementation of different components of the system, but the interested reader can find this elsewhere [1].

VI. TIMING DEPENDENCIES

For simplicity, we will consider only the input (ip), context (cxt), delay (del) and data store (store) layers, which are sufficient to achieve the basic implementation of the sequence machine. The spike bursts from these layers have to observe certain time constraints to enable the system to function.

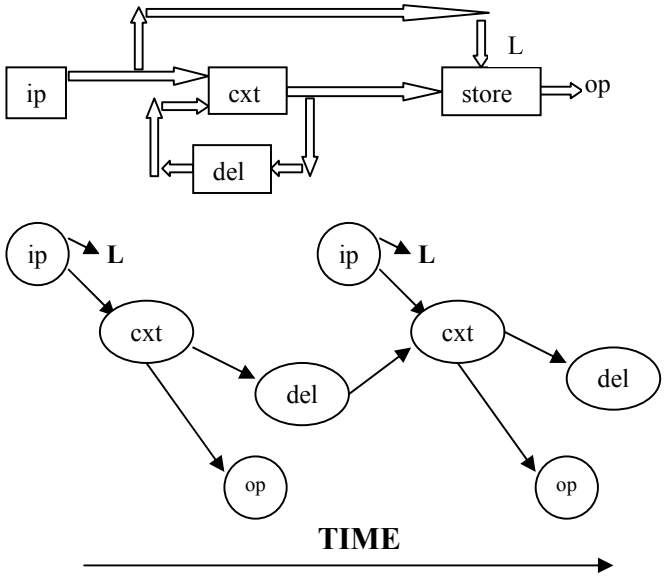

Fig. 2. The primary components of the system and their timing dependencies. Both the inputs to the cxt layer increase only the activation of the neurons, while the store layer has a normal input from the cxt layer to increase the activation, and a special learning input L which signals writing of the association of the ip and the cxt to the data store memory.

The timing dependencies between the bursts from different layers in the system can be summarised as follows:

1. The two input bursts to the context layer (fed back old context from the delay layer and the new input from the input layer) have to be approximately coincident, else the context neurons could start firing before receiving all the inputs, which would destroy the code transmitted by the burst, which is in the rank of the spikes. Therefore, the delay time (which is internal to the system) has to be matched to the gap between different inputs (which is external to the system).
2. The outputs of the store layer, which form the prediction of the next inputs to the system, must come before the next inputs to the system. So the gap between inputs should be bigger than the time taken for a spike burst to propagate through all layers of the system in the forward direction (excluding the

feedback through the delay layer).

3. The store neurons function in two modes: the normal or recall mode, in which they get input spikes from the context layer which increase the activations, and the learning mode, in which the association of the past context and the new input gets written to the memory. The learning mode gets triggered by the firing of the learning spikes from the input layer. We need to store the order of the context burst spikes until the next input burst comes and the association can be written to the memory. We do so by storing the order in the synapses of the neurons.
4. Also, in the learning mode, we have to make sure that the store neurons receive all the spikes from learning inputs (and complete writing the association) before receiving the spikes from the context (which are to be stored for the next association, when the new input burst comes). The latency of the context layer can ensure this.
5. Initially, the delay layer has no inputs (because there is no previous context) and consequently the context layer will fire slower (since its delay inputs are missing) than it would normally. We have to ensure that this does not destabilise the system, and the inter-burst interval stabilises after passing through a few layers.

VII. OTHER ISSUES

There are some other issues concerning implementation by asynchronous spiking neurons in general as well as some issues specific to the sequence machine system, which we have to deal with. They are briefly summarised below.

1. We need a signal to indicate the beginning and end of a burst, because all the neurons in a layer reset their phases when the burst begins. We consider the first input spike as the beginning of the burst to reset the phase of all neurons, and the output of the counter signifying that N neurons in that layer have fired, which is the maximum permissible according to the N-of-M code.
2. We need to store the rank ordering of the two input bursts to the context (from the delay and input layers) separately, since the increase of activation of the context on receiving any input depends on the position of that input in its respective burst. This is done by having two different feed-forward desensitisation neurons on the two kinds of inputs to the context, which keep track of the rank of the input spikes from both the layers.
3. The bursts of spikes have to be stable (not blow up or die out) and coherent (not interfere with each other, and clearly separated) as they pass through different neural layers in the system. This can be achieved by using a combination of feed-forward and feedback inhibition, as shown earlier [2].
4. We have to ensure that output spikes of any layer do not start firing before it has received all the input spikes from the layer before it, else this will spoil the code being transmitted. This can be arranged by having large thresholds and axonal or wire delays to ensure that this case does not happen.
5. We assume for now that the danger of the system being caught waiting forever will not happen, as long as the bursts are stable and coherent as described above.
6. The system is very sensitive to noise in the spike trains and so needs to be carefully engineered so that the times of firing are precise. However there is a degree of redundancy gained from using ordered N-of-M code, as the number of actual codes used in the alphabet is far less compared to the total possible number of codes, so some error is tolerable.
7. It is better to have the latencies (average time between the input and output bursts) of each layer comparable, in order to increase the stability of the system as a whole. To enable this, layers with more inputs should have higher thresholds and vice versa, because more inputs mean that the activations will rise quicker and the intra-burst separation will be small for that layer.
8. In our implementation of the system using spiking neurons, we have to ensure that the output spike burst from a layer in response to an input burst is equivalent (with respect to the code being considered, i.e. the rank and choice of neurons firing in this case) to what we would expect in the high-level model we are implementing (using ordered N-of-M coded symbols). The wheel model of spiking neurons that we have chosen meets this requirement.
9. The layers have control over their output spikes, but have no knowledge of their input spikes (unless each layer asks the layer before it). Therefore we have to ensure that the output spikes follow the correct code and there are no errors in generation of the output spikes, else the neurons in the next layer could keep waiting indefinitely for the expected number of input spikes. Having an N-of-M code solves this problem, as it is self error-correcting.
10. The robustness of the system depends on the stability of the bursts, so bursts emitted by different layers and the same layer during different waves should be well separated in time. Similarly, the inter-burst and intra-burst time separations should remain stable.

To ensure that the system works as planned, we have to use certain control mechanisms as discussed (while not compromising on the requirement that there should be no global variables in the system and neurons should fire based only on input spikes), but we have to minimise them as much as possible, in order to increase the flexibility of the system.

VIII. SIMULATION

We used a spiking neural simulator developed by M. Cumpstey [3] to simulate the complete system. The simulator is generic, event-driven, object-oriented and suitable for most common spiking neural models. We specify the network

configuration and the simulation file, and the simulator outputs a series of spikes from different layers along with their time of spiking. We had to make a few changes to the original simulator to incorporate some of the issues discussed.

Below is a diagram of the output of the simulator (with spike outputs of different neural layers against time) on being given a repeated input sequence 715171517151. The first time the sequence 7151 is given the output prediction is incorrect, but the system learns to predict correctly and the next time the prediction is correct. The sequence 7151 is used because it is the simplest sequence where we need to have knowledge of context to determine the successor of the symbol "1".

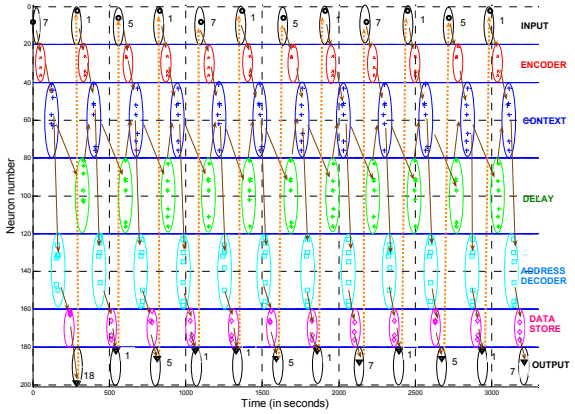

Fig. 3. Plot of spikes emitted by different layers in the sequence machine against time. The arrows denote causality, how a burst of spikes causes firing of another burst in the next layer after a delay. Spikes of the same shape and in parallel vertical bands belong to the same layer, and ellipses enclose bursts of spikes. The figure plots 12 different waves of spike bursts each triggered by an input spike, and forming the sequence 715171517151. After the first 7151 input when the system learns the sequence, the prediction of 15171517 on the output is correct.

In figure 3, spikes from different layers are plotted on the Y-axis and time on the X-axis. Spikes of the same colour belong to the same layer, and the arrows show how a burst of spikes from one layer causes the next layer to fire a burst after some time. We can see from the diagram that the spike bursts from different layers are coherent, stable, well behaved, and follow the timing dependencies mentioned. They also implement the high-level sequence machine by learning the given sequence 7151 in a single pass, and predicting it correctly in the second and third presentation of 7151.

IX. Conclusion and Future Work

We have shown that it is possible to build a system out of asynchronous spiking neurons that can perform the high-level functionality of the sequence machine.

We have mentioned how common components in traditional asynchronous design, such as the latch and the handshake protocol, might be implemented in spiking neurons. If we adapt traditional asynchronous design components such as latches to take into the additional constraints of spiking neural implementation, we could potentially reach a stage where it is possible to translate any spiking neural network into its equivalent asynchronous logic circuit, enabling us to make use of the synthesis tools available in asynchronous logic design to analyse a spiking neural network.

References

[1] An associative memory for the on-line recognition and prediction of temporal sequences, by J. Bose, S. B. Furber and J. L. Shapiro, in proceedings of International Joint Conference of Neural Networks (IJCNN 2005), Montreal, Canada, 31 July - 4 August 2005.

[2] A system for transmitting a coherent burst of activity through a network of spiking neurons, by J. Bose, S. B. Furber and J. L. Shapiro, in proceedings of 16th Italian workshop on Neural nets (WIRN 2005), Vietri sul Mare, Italy, 8-11 June 2005.

[3] SpikeNetwork: A spiking neural simulator, by M. Cumpstey, private communication.